69. Цикл for

Безусловно, наиболее используемым циклом в языке C++ является цикл for.

Оглавление:

1. Цикл for
2. Выполнение цикла for
3. Еще примеры циклов for
4. Ошибка неучтенной единицы
5. Пропущенные выражения в цикле
6. Объявление переменных в цикле for
7. Использование цикла for в старых версиях C++
8. Вложенные циклы for
9. Заключение
10. Тест

Цикл for

Цикл for в языке C++ сделан, когда известно необходимое количество операций. Выглядит он следующим образом:

for (объявление переменных; условие; инкремент/декремент счетчика)
    тело цикла;

Или, преобразуя for в эквивалентный цикл while:

```
{ // объявляем переменные, цикл находится в блоке
    объявление переменных;
    while (условие)
    {
        тело цикла;
        инкремент/декремент счетчика;
    }
} // переменные, объявленные внутри цикла, выходят из области видимости здесь
```

Переменные, определенные внутри цикла for, имеют специальную **локальную область видимости**, называемую **область видимости цикла**. Такие переменные существуют только внутри цикла и недоступны за его пределами.

Выполнение цикла for

Цикл for в C++ выполняется в 3 шага:

Шаг №1. Объявление переменных. Как правило, здесь выполняется определение и инициализация счетчиков цикла, а точнее — одного счетчика цикла. Это часть выполняется только один раз, когда цикл выполняется впервые.

Шаг №2. Условие. Если оно равно false, то цикл немедленно завершает свое выполнение. Если же условие равно true, то выполняется тело цикла.

Шаг №3. Инкремент/декремент счетчика цикла. Переменная увеличивается или уменьшается на единицу. После этого цикл возвращается к шагу №2.

Рассмотрим пример цикла for и разберем детально, как он работает:

```
// Выводим числа от 1 до 5
int main()
{
    for (int count = 1; count <= 10; ++count)
        std::cout << count << ' ';

    return 0;
}
```

Сначала мы объявляем переменную count и проговариваем ей значение 1. Далее проверяется условие count <= 10, а так как count равен 1, то условие 0 <= 10 имеет значение true. Следовательно, выполняется тело цикла, в котором мы выводим в консоль переменную count (т.е. значение 1).

Затем выполняется выражение ++count, т.е. инкремент переменной. Затем цикл снова возвращается к проверке условия. Условие 2 <= 10 имеет значение true, поэтому тело цикла выполняется опять. Выводится 2, а переменная count увеличивается ещё до значения 3. Условие 3 <= 10 является истинным, поэтому выводится 3, а count увеличивается до 3 и так далее.

В конце концов, count увеличивается до 10, а условие 10 <= 10 является истинным, и цикл завершается. Следовательно, результат выполнения программы:

```
1 2 3 4 5 6 7 8 9 10
```

Цикл for могут быть несколько сложны для новичков, однако опытные кодеры любят их, так как эти циклы очень компактны и удобны. Для наглядности давайте преобразуем цикл for, приведенный выше, в эквивалентный цикл while:

```
// Выводим числа от 1 до 5
int main()
{
    // Эквивалентный цикл while для объявления переменных
    int count = 1;
    while (count <= 10)
    {
        std::cout << count << ' ';
        ++count;
    }

    return 0;
}
```

Обратите внимание, внешние фигурные скобки здесь необходимы, так как переменная count выходит из области видимости при завершении цикла.

Еще примеры циклов for

Давайте, используя цикл for, напишем функцию вычисления значений в степени x:

```
// Для простоты пока base, exponent и count - глобальные переменные
int base, exponent, count;

int pow(int base, int exponent)
{
    int total = 1;

    for (int count = 0; count < exponent; ++count)
        total *= base;

    return total;
}
```

Функция возвращает значение base^exponent (число в степени x). Base — это число, которое нужно возвести в степень, а exponent — это степень, в которую нужно возвести base:

- Если экспонент равен 0, то цикл for выполняется 0 раз и функция возвращает 1.
- Если экспонент равен 1, то цикл for выполняется 1 раз и функция возвращает 1 * base.
- Если экспонент равен 2, то цикл for выполняется 2 раза, и функция возвращает 1 * base * base.

Хотя в большинстве циклов используется инкремент счетчика, мы также можем использовать и декремент счетчика:

```
// Выводим числа от 10 до 1
int main()
{
    for (int count = 10; count > 0; --count)
        std::cout << count << ' ';

    return 0;
}
```

Результат:

```
10 9 8 7 6 5 4 3 2 1
```

Также с каждой новой операцией мы можем увеличить или уменьшить значение счетчика больше, чем на единицу:

```
// Выводим нечетные числа от 1 до 9
int main()
{
    for (int count = 1; count <= 9; count += 2)
        std::cout << count << ' ';

    return 0;
}
```

Результат:

```
1 3 5 7 9
```

Ошибка неучтенной единицы

Один из самых больших проблем с циклом for приходится сталкиваться начинающим программистам в цикле for (а также и в других типах циклов) — это **ошибка на единицу** (или **ошибка неучтенной единицы**). Она возникает, когда цикл повторяется на 1 раз больше или на 1 раз меньше нужного количества итераций. Это обычно происходит из-за того, что в условии используется неверный оператор сравнения (например, > вместо >= или наоборот). Как правило, эта ошибка приводит к тому, что цикл выполняется на один дополнительный или, программа будет работать, но её результаты будут неправильными.

При написании циклов for помните, что цикл будет выполняться до тех пор, пока условие является истинным. Рекомендуется тестировать циклы, используя разные значения для проверки работоспособности цикла. Хорошей практикой является проверять циклы с помощью данных ввода (числа, символы и прочего), которые заставляют цикл выполняться 0, 1 и 2 раза. Если цикл работает нормально, значит всё ОК.

Правило: Тестируйте свои циклы, используя входные данные, которые заставляют цикл выполняться 0, 1 и 2 раза.

Пропущенные выражения в цикле

Также в цикле можно пропускать одно или сразу все выражения, например:

```
int main()
{
    int count = 0;
    for (; count < 10; )
    {
        std::cout << count << ' ';
        ++count;
    }

    return 0;
}
```

Результат:

```
0 1 2 3 4 5 6 7 8 9
```

Инициализация счетчика мы перенесли вне тела цикла, а инкремент счетчика — внутри тела цикла. В самом операторе for мы указали только условие. Иногда бывают случаи, когда не требуется объявлять счетчик цикла (потому что у нас он уже есть) или увеличивать его (так как мы увеличиваем его каким-то другим способом).

Хотя это и не часто можно наблюдать, но в операторе for можно вообще ничего не указывать. Стоит отметить, что подобное приводит к бесконечному циклу:

```
for (;;)
    тело цикла;
```

Вышеуправленный пример эквивалентен:

```
while (true)
    тело цикла;
```

Объявление переменных в цикле for

Хотя в цикле for обычно используется только один счетчик, иногда могут возникнуть ситуации, когда нужно работать сразу с несколькими переменными. Для этого используется оператор запятая. Например:

```
// Выводим числа от 0 до 9 и числа от 100 до 99
int main()
{
    for (int x, y(100); x < 10; ++x, --y)
        std::cout << x << ' ' << y << ' ';

    return 0;
}
```

Этот цикл показывает значения двух разных объявленных переменных: int x = 0 и int y = 9. Только с каждой операцией переменная x увеличивается на единицу, а переменная y — уменьшается на единицу.

Результат выполнения программы:

```
0 99
1 98
2 97
3 96
4 95
5 94
6 93
7 92
8 91
9 90
```

Примечание: Вышеупомянутый цикл можно переписать следующим образом:

```
// Выводим числа от 0 до 9 и числа от 100 до 99
int main()
{
    int x, y(100);
    for (x = 0; x < 10; ++x, --y)
        std::cout << x << ' ' << y << ' ';

    return 0;
}
```

В этом случае запятая в объявлении переменных является частью синтаксиса, а не использованием оператора Запятая. Но эффект идентичен.

Использование цикла for в старых версиях C++

В старых версиях C++ переменные, объявленные в цикле for, не уничтожались при завершении этого цикла. Т.е. у вас могло получиться что-то вроде следующего:

```
// Если компилятор использует старые версии C++
for (int count = 0; count < 10; ++count)
    std::cout << count << ' ';

// Если компилятор использует старые версии C++, переменная count по-прежнему существует здесь
std::cout << "Hello!\n";
std::cout << "Переменная count по-прежнему существует здесь: " << count << '\n';
```

Поэтому такие программы, на вы или еще можете увидеть подобное в старом коде.

Вложенные циклы for

Подобно другим типам циклов, один цикл for могут быть вложены в другие циклы for. В следующем примере мы развернули один for внутри другого for:

```
// Выводим 4 строки по 5 символов каждая
int main()
{
    for (int i = 'A'; i <= 'Y'; ++i)
    {
        std::cout << i << ' ';// Выводим первый символ
        for (int k = 0; k < 3; ++k)// Выводим следующие три символа
            std::cout << (char) (i + k);
        std::cout << '\n';
    }

    return 0;
}
```

С одной операцией внешнего цикла выполняются три операции внутреннего цикла. Следовательно, результат выполнения программы:

```
ABCD
BCDE
CDEF
DEFG
EFGH
```

Заключение

Циклы for являются наиболее часто используемыми циклами в языке C++. Несмотря на то, что их синтаксис, как правило, немного запутывает начинающих программистов, вы очень скоро к нему привыкнете и поймете все между и удобство этих циклов.

Тест

Задание №1

Напишите цикл for, который выводит каждое четное число в диапазоне от 0 до 20.

Задание №2

Напишите функцию sum(int), которая принимает целочисленный параметр с именем value и возвращает сумму всех чисел от 1 до значения value.

Например, если значение value является 3, то sum(3) должен возвратить 1 + 2 + 3 = 6.

Подсказка: Используйте переменную вне тела цикла для хранения суммы чисел, как в примере с функцией pow() в подразделе «Еще примеры циклов for».

Задание №3

Что не так со следующим циклом?

```
// Выводим числа от 1 до 10
for (unsigned int count = 9; count >= 0; --count)
    std::cout << count << ' ';
```

Ответ №1

```
// Выводим четные числа от 0 до 20
int main()
{
    for (int count = 0; count <= 20; count += 2)
        std::cout << count << std::endl;

    return 0;
}
```

Ответ №2

```
int sum(int value)
{
    int total(0);
    for (int count(1); count <= value; ++count)
        total += count;

    return total;
}
```

Ответ №3

Этот цикл for выполняется до тех пор, пока count >= 0. Другими словами, он работает до тех пор, пока переменная count не станет отрицательным числом. Однако, поскольку count является типа unsigned, то эта переменная никогда не сможет быть отрицательной. Следовательно, этот цикл бесконечный! Как правило, рекомендуется избегать использования типов unsigned в циклах, если вы это нет веских причин.